

PROJECT REPORT
ON
HOTEL TABLE BOOKING
RESERVATION SYSTEM

**Submitted in partial fulfilment of the
requirement for the award of the degree of
Bachelor of Computer Applications
to
Uttaranchal School of Computing Sciences**



**UTTARANCHAL
UNIVERSITY
DEHRADUN**

**NAAC
GRADE A+**

SESSION (2025-26)

Submitted To:

Mr. Sahil Gupta

Assistant Professor

USCS

Submitted By:

Vishal Panwar

BCA-4th B

UU2409000349

ACKNOWLEDGEMENT

The most awaited moment of any endeavor is successful completion, but nothing can be done successfully if done alone. Success is the outcome of contribution and consistent help of various persons and we thank those ones who helped us in successful completion of this project.

First and foremost, we would like to thanks **Prof. (Dr.) Sonal Sharma**, Director, **USCS** for providing a healthy and encouraging environment to study.

We profusely thankful to the “**Prof. (Dr .) Sameer Dev Sharma**”, Head of Department, **USCS** and our project Mentor **Mr. Sahil Gupta** been generous enough to provide us an opportunity and accepting our candidature for the most valuable guidance and affordable treatment given to us at every stage to boost our morale.

We express our sincere gratitude to our colleagues with whom we have fruitful discussions which have helped us a lot in giving a final shape to the program.

Date:

Name and Signature:

Enrollment No- UU2409000349

Vishal Panwar

DECLARATION

I, Mr. Vishal Panwar, S/o Mr. Vijay Chaudhary, bearing Enrollment No. UU2409000349 student of **program BCA** semester 4th hereby declare that the project report entitled **Hotel Table Booking Reservation System** submitted to the **USCS, Uttaranchal University, Dehradun**, is a record of an original work carried by me under the guidance of Mr. Sahil Gupta and this project work has not been submitted to any other University / College for the sake of award of any degree.

Date:

Student Name and Signature

Vishal Panwar

CERTIFICATE OF ORIGINALITY

This is to certify **Vishal Panwar** student of BCA 4th Semester, of **Uttaranchal School of Computing Sciences, Dehradun**, has completed the Project Report for the topic Hotel Table Booking Reservation System for the session (2025-26).

Under the Guidance of :

Mr . Sahil Gupta

Dept. of Computing Sciences

USCS, Uttaranchal University

TABLE OF CONTENT

ACKNOWLEDGEMENT	ii
DECLARATION	iii
CERTIFICATE OF ORIGINALITY	iv
INTRODUCTION	1
OBJECTIVE	2
SYSTEM ANALYSIS	3
IDENTIFICATION OF NEED	6
PRELIMINARY INVESTIGATION	7
FEASIBILITY STUDY	9
PROJECT PLANNING	11
GANTT CHART	14
PERT CHART	15
SOFTWARE REQUIREMENT SPECIFICATION	16
SYSTEM ENGINEERING PARADIGM APPLIED	19
DATA FLOW DIAGRAM	21
ER-DIAGRAM	22
FLOWCHART	23
SYSTEM DESIGN	24
MODULARISATION DETAILS	26
DATA INTEGRITY AND CONSTRAINTS	28
DATABASE DESIGN	31
USER INTERFACE DESIGN	33
TESTING	36
SYSTEM SECURITY MEASURES	39
FUTURE SCOPE AND FURTHER ENHANCEMENT	41
CODING	42
BIBLIOGRAPHY & PROFILES OF TWO IT COMPANIES	48

INTRODUCTION

The **Hotel Table Booking Reservation System** is a web-based application designed to streamline the process of reserving tables, placing food orders, and managing restaurant operations efficiently. With the increasing demand for digital convenience, traditional manual booking and order management systems are becoming inefficient, error-prone, and difficult to monitor in real time. This project aims to provide a structured and automated solution that improves both customer experience and administrative control.

The system allows users to select tables based on availability, book them for a specific date and time, and place food orders through an interactive menu interface. The selected items, along with quantities and pricing, are processed and stored in a centralized MySQL database. Additionally, a payment module enables tracking of the total amount paid by each table, ensuring proper financial management and transparency.

On the administrative side, the system includes a dashboard that provides a complete overview of all bookings, including customer details, table allocation, ordered food, order status, and payment information. This enables efficient monitoring of restaurant activity, reduces manual workload, and minimizes the chances of booking conflicts or data inconsistency.

The backend is developed using Java Servlets, which handle data processing, database communication, and business logic, while the frontend is built using HTML, CSS, and JavaScript to provide a responsive and user-friendly interface. By integrating booking, ordering, and payment tracking into a single system, the project demonstrates how web technologies can be used to build a practical and scalable restaurant management solution.

OBJECTIVE

- To develop a web-based Hotel Table Booking Reservation System that allows users to efficiently reserve tables by selecting preferred date, time, and seating capacity.
- To design and implement an interactive food ordering module that enables users to browse menu items, select quantities, and place orders in a structured manner.
- To integrate a backend system using Java Servlets and MySQL for processing booking data, storing order details, and maintaining consistency in database operations.
- To provide an administrative dashboard that allows real-time monitoring of table bookings, customer details, ordered food items, order status, and payment records.
- To ensure smooth coordination between booking, ordering, and payment modules, thereby reducing manual errors, avoiding double bookings, and improving overall system efficiency.
- To demonstrate the practical application of web technologies such as HTML, CSS, JavaScript, Java Servlets, and MySQL in building a functional and scalable restaurant management system.

SYSTEM ANALYSIS

1. Problem Definition

- In today's fast-paced digital environment, traditional restaurant management systems that rely on manual booking and order handling are inefficient, error-prone, and difficult to manage. Customers often face issues such as unavailability of tables, lack of real-time booking information, and delays in order processing. At the same time, restaurant staff struggle with maintaining records, avoiding double bookings, and tracking customer orders and payments accurately..
- Most small-scale systems do not provide a unified platform where table reservation, food ordering, and payment tracking are integrated together. This leads to mismanagement, data inconsistency, and poor customer experience.
- The **Hotel Table Booking Reservation System** addresses these problems by providing a centralized digital platform that allows users to book tables, place food orders, and manage payments efficiently. It also includes an admin module that helps monitor bookings, orders, and payments in real time, ensuring better coordination and system reliability..

2. System Requirements

The requirements are divided into **functional** and **non-functional**:

1. Functional Requirements

- User can book tables by selecting date, time, and number of guests.
- System prevents double booking of the same table at the same time.
- Users can select food items from a menu and place orders.
- System calculates total bill based on selected items and quantity.
- Payment amount is recorded and linked to each booking.
- Admin can view all bookings with details (table, customer, food, status, payment).
- Admin dashboard displays real-time data of all reservations and orders.
- System maintains history of bookings and orders in MySQL database.

2. Non- Functional Requirements

- Fast response time for booking and ordering operations.
- Reliable database management ensuring data consistency.
- User-friendly interface for both customers and admin.
- Secure backend processing using Java Servlets.
- Scalability to handle multiple users simultaneously.
- Maintainability for future upgrades (UI improvements, new features).

3. Feasibility Study

1. Technical Feasibility

- Developed using Java Servlets for backend logic.
- MySQL database for storing booking, order, and payment data.
- HTML, CSS, and JavaScript for frontend interface.
- REST-based communication between frontend and backend using fetch API.

2. Operational Feasibility

- Easy-to-use interface for booking and ordering.
- Reduces manual effort for both customers and staff.
- Admin dashboard simplifies monitoring of all activities..

3. Economic Feasibility

- Uses open-source technologies (Java, MySQL).
- No licensing cost involved.
- Low maintenance cost as system updates are mainly backend-based.

4. System Objectives

- To provide an efficient digital system for table booking and management..
- To allow users to place food orders easily through an interactive menu..
- To maintain accurate records of bookings, orders, and payments..
- To prevent conflicts such as double booking of tables.
- To provide real-time monitoring through an admin dashboard..

5. Data Flow Analysis

1. Input Stage

- User enters booking details (name, guests, date, time)
- User selects a table based on availability.
- User chooses food items and quantity from menu.
- Booking and order data are sent to backend server.

2. Processing Stage

- Validate table availability.
- Store booking details in database..
- Calculate total bill based on selected items.
- Update food and payment information in database.
- Manage latest booking records using backend logic.

3. Output Stage

- Booking confirmation displayed to user.
- Food order and bill displayed on bill page.
- Admin dashboard displays:
 - Total bookings
 - Customer details
 - Ordered food
 - Order status
 - Payment amount

IDENTIFICATION OF NEED

1. Increasing Demand for Digital Booking Systems:

With the growing use of online services, customers prefer digital solutions for reserving tables instead of manual or phone-based bookings, which are time-consuming and inefficient..

2. Limitations of Traditional Booking Methods:

Manual table reservation systems often lead to errors such as double bookings, miscommunication, and lack of proper record management, creating inconvenience for both customers and staff.

3. Lack of Integrated Ordering System:

Most small restaurant systems do not provide a unified platform where users can book tables and place food orders together, resulting in fragmented operations.

4. Absence of Real-Time Table Availability:

Customers are unable to check which tables are available at a specific time, leading to confusion and poor user experience.

5. Need for Efficient Payment Tracking:

Restaurants require a system that records and tracks payment details for each booking to maintain financial accuracy and avoid manual calculation errors.

6. Requirement for Centralized Data Management:

A structured database system is needed to store booking details, food orders, and payment records securely for easy retrieval and management.

7. Need for Admin Monitoring and Control:

An admin interface is essential to monitor all bookings, customer details, ordered food items, and payments in real time, ensuring smooth restaurant operations.

8. Reduction of Human Errors and Workload:

Automating booking, ordering, and payment processes reduces manual workload, minimizes errors, and improves overall efficiency.

9. Improved Customer Experience:

The system enhances user convenience by providing a simple interface for booking tables, ordering food, and viewing bills, leading to faster and more reliable service.

PRELIMINARY INVESTIGATION

1. Introduction

Before developing any web-based management system, it is important to conduct a preliminary investigation to understand the limitations of existing manual processes and identify the need for automation. For the **Hotel Table Booking Reservation System**, this investigation focuses on the inefficiencies of traditional restaurant operations, such as manual table booking, lack of proper order tracking, and difficulty in managing payments.

The study highlights how a centralized digital system integrating booking, ordering, and payment tracking can improve accuracy, reduce workload, and enhance overall user experience.

2. Current System Overview

Currently, many restaurants rely on manual or partially digital systems, which suffer from the following limitations:

- Table reservations are handled manually, leading to errors and confusion
- High chances of double booking due to lack of real-time availability tracking.
- Food orders are not systematically recorded, causing mismanagement.
- No proper tracking of payment details for each table.
- Lack of centralized database for storing booking and order information
- No admin dashboard for monitoring all activities in one place.
- Time-consuming process for both customers and staff.

3. Proposed System

The proposed solution is a **Hotel Table Booking Reservation System** developed using Java Servlets for backend processing and MySQL for database management. The system will:

- Allow users to book tables by selecting date, time, and number of guests..
- Prevent double booking of the same table using validation logic.
- Provide an interactive menu for selecting food items and quantities.
- Calculate total bill automatically based on selected items.
- Store booking, order, and payment details in a MySQL database.

- Maintain proper linkage between table, food order, and payment.
- Provide an admin dashboard to view all bookings, orders, and payments.
- Display real-time data for better monitoring and decision-making.

4. Advantages of Proposed System

- **Efficiency:** Reduces manual work and speeds up booking and ordering processes.
- **Accuracy:** Minimizes errors such as double booking and incorrect billing
- **Centralized Data:** Stores all information in a structured database for easy access.
- **Real-Time Monitoring:** Admin can track bookings, orders, and payments instantly.
- **User-Friendly:** Simple interface for customers to book tables and place orders
- **Scalability:** Can be extended with additional features like online payment or mobile integration.
- **Better Management:** Improves coordination between booking, ordering, and payment processes.

FEASIBILITY STUDY

1. Technical Feasibility

- The system is developed using Java Servlets for backend processing and MySQL for database management.
- HTML, CSS, and JavaScript are used to build a responsive and user-friendly frontend interface.
- The system follows a structured client-server architecture, ensuring smooth communication between frontend and backend using HTTP requests (Fetch API).
- MySQL database securely stores booking details, food orders, and payment records in a structured format.
- The system is modular in design, allowing easy updates and future enhancements (such as adding online payment or mobile integration).

2. Operational Feasibility

- The system is designed to be simple and easy to use for both customers and administrators.
- Users can quickly book tables, select food items, and view bills without requiring technical knowledge.
- The system automates table booking, order management, and payment tracking, reducing manual effort.
- Admin users can monitor all bookings, orders, and payments through a centralized dashboard.
- The web-based nature of the system allows easy access through any browser without installing additional software.

3. Economic Feasibility

- The project uses open-source technologies such as Java, MySQL, HTML, CSS, and JavaScript, making it cost-effective.
- No licensing cost is required for development or deployment.

- Maintenance cost is low, as updates mainly involve improving features or UI
- The system can be scaled with minimal additional investment.

4. Legal Feasibility

- **Data Protection Compliance:** The system stores only necessary data such as customer name, booking details, and order information. No sensitive personal or financial data is collected.
- **No Licensing Issues:** The system is built using open-source technologies, ensuring compliance with software licensing standards.
- **Intellectual Property:** The system design, logic, and implementation are original, avoiding any copyright issues.

5. Ethical Feasibility

- **User Privacy:** User data is securely stored and used only for booking, ordering, and payment tracking purposes.
- **Transparency:** The system clearly displays booking details, ordered items, and payment amounts to users.
- **Fair Usage:** The system prevents double booking and ensures equal access to available tables.
- **Data Confidentiality:** User and booking data are not shared with any third parties, maintaining privacy and trust.

PROJECT PLANNING

1. Project Objectives

- To develop a web-based system for efficient table booking and reservation management.
- To provide an interactive menu system for selecting food items and placing orders.
- To implement backend processing using Java Servlets for handling bookings, orders, and payments.
- To maintain a centralized database using MySQL for storing booking details, food orders, and payment records.
- To provide an admin dashboard for monitoring all bookings, orders, and payment information in real time.

2. Project Scope

- Developing a Java Servlet + MySQL based backend system for booking and order management.
- Designing frontend pages using HTML, CSS, and JavaScript for user interaction.
- Providing features such as:
 - Table booking with date, time, and guest selection
 - Food ordering through a structured menu interface
 - Automatic bill calculation and payment tracking
- Enabling admin functionalities such as viewing all bookings, orders, and payment details.
- Ensuring proper database management for storing and retrieving all system data.

3. Timeline

- Requirement Gathering: Identify system features, booking logic, and database design – 1 week
- System Design: Define architecture, UI flow, and ER/DFD diagrams – 1 week
- Backend Development: Implement Java Servlets (booking, food, payment, admin) – 3 weeks
- Database Setup: Design and create MySQL tables (Bookings, Orders, Payments) – 1 week
- Testing: Debug backend logic, validate data flow, and test all modules – 1 week
- Integration: Connect frontend with backend and ensure smooth operation – 1 week
- Documentation: Prepare report, diagrams, and project explanation – 1 week
- Deployment: Final testing and system execution – 1 week

4. Resource Allocation

Resources required for successful completion:

Software Tools:

- Java (Servlets for backend processing)
- MySQL database server
- HTML, CSS, JavaScript for frontend development

Hardware Requirements:

- Laptop or PC with Java and MySQL installed
- Web browser for testing (Chrome/Edge)
- Internet connection for development and testing

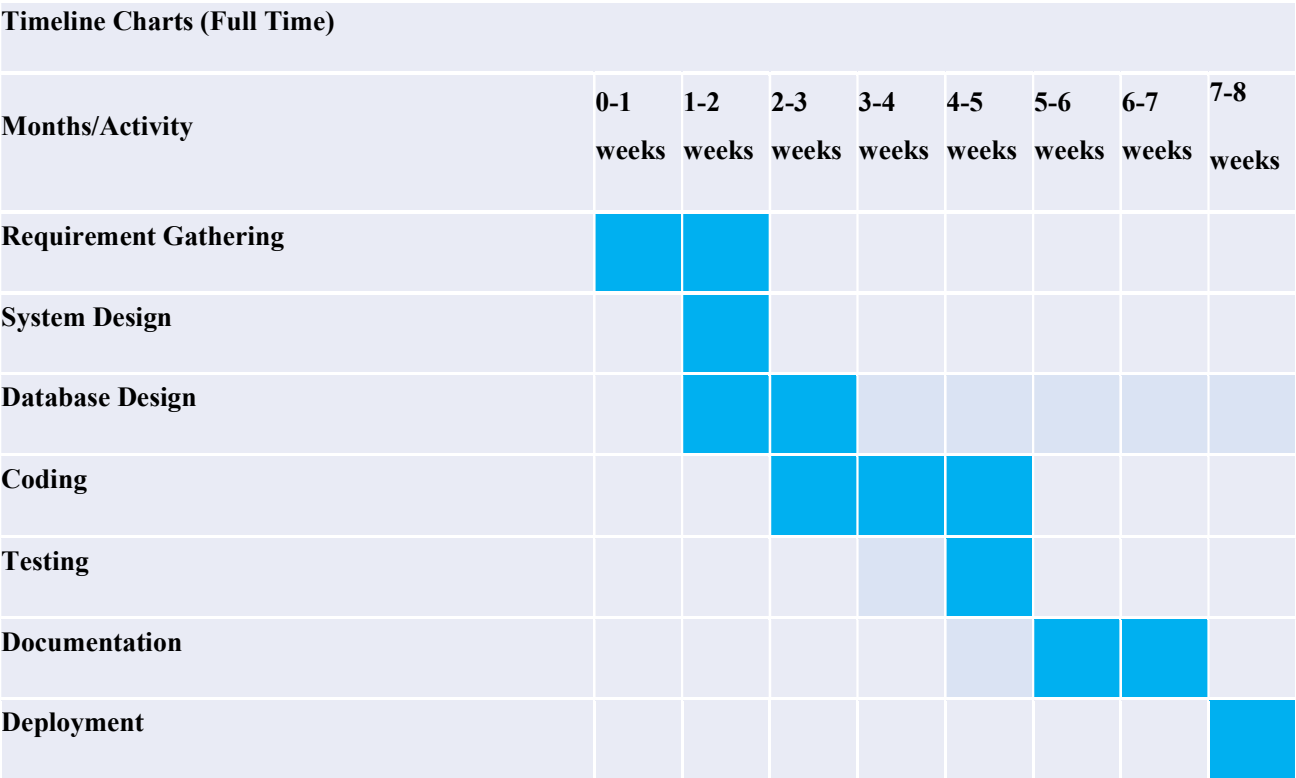
5. Risk Management and Mitigation

Potential risks and strategies:

- **Double Booking Issue:** Implement validation logic to prevent booking the same table at the same date and time.
- **Data Loss:** Regular backups of MySQL database to protect booking and payment data.

- **Incorrect Order Handling:** Validate user input and ensure correct mapping between table and food orders.
- **Performance Issues:** Optimize database queries and backend processing for faster response.
- **System Errors:** Perform thorough testing of booking, ordering, and admin modules.
- **Scalability:** Design modular code structure to allow future enhancements such as online payment and mobile access.

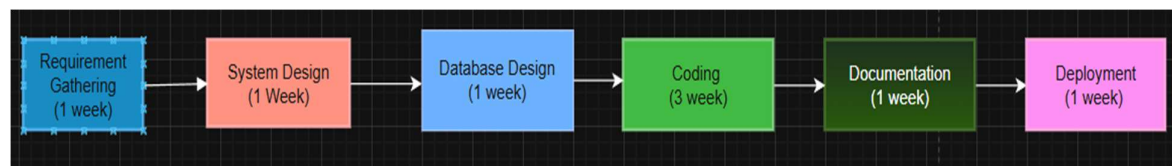
GANTT CHART



PERT CHART

Activity id	Activity Name	Duration(weeks)	Depends on
A	Requirement Gathering	1	-
B	System Design	1	A
C	Database Design	1	B
D	Coding	3	C
E	Testing	1	D
F	Documentation	1	E
G	Deployment	1	F

Diagram:



SOFTWARE REQUIREMENT SPECIFICATION

1. Introduction

The **Hotel Table Booking Reservation System** is a web-based application designed to manage restaurant table reservations, food ordering, and payment tracking in an efficient and structured manner. The system allows users to book tables based on availability, select food items from an interactive menu, and generate bills automatically.

The application maintains all booking, order, and payment data in a MySQL database and provides an admin dashboard for monitoring all activities in real time. The main objective of the system is to reduce manual work, avoid booking conflicts, and improve overall customer experience through digital automation.

2. Functional Requirements

User Management:

- **Table Booking:** Users can select table, date, time, and number of guests.
- **Booking Validation:** System prevents double booking of the same table at the same time.

Food Order Management

- **Menu Selection:** Users can view menu items with price and images.
- **Order Placement:** Users can select items and quantity to place an order.
- **Bill Calculation:** System calculates total amount based on selected items.

Admin Management

- **View Bookings:** Admin can view all table bookings with details.
- **View Orders:** Admin can see ordered food items for each table.
- **Payment Tracking:** Admin can monitor payment details of each booking.
- **Dashboard View:** Displays complete system data in tabular format.

Payment Management

- **Payment Recording:** System stores total payment amount for each booking.
- **Bill Display:** Users can view ordered items and total bill amount.

3. Non-Functional Requirements

User Interface

- Simple, clean, and user-friendly interface.
- Clear display of booking details, food items, and payment information..

Performance

- Booking and order processing should be completed within a few seconds.
- System should handle multiple users simultaneously without delay.

Portability

- Compatible with Windows operating system.
- Accessible through web browsers (Chrome, Edge, etc.).
- Can be extended to mobile-based interfaces in future.

Security

- Input validation to prevent invalid or incorrect data entry.
- Protection against SQL injection through prepared statements..
- Secure handling of booking and payment data.

4. Software Requirements

Hardware Requirements

- Processor: Minimum 2.0 GHz dual-core processor
- Memory: At least 2 GB RAM
- Storage: 200 MB free disk space

Software Requirements

- Backend: Java (Servlets)
- Database: MySQL
- Frontend: HTML, CSS, JavaScript.
- Development Tools: VS Code / MySQL Workbench
- Server: Apache Tomcat
- OS Compatibility: Windows

SYSTEM ENGINEERING PARADIGM APPLIED

To develop the **Hotel Table Booking Reservation System**, the **Waterfall Model with Iterative Feedback** is applied. This approach ensures a structured, step-by-step development process while allowing improvements based on testing and feedback. The stages are:

1. System Requirement Analysis

- Identified the need for a digital system to manage table booking, food ordering, and payment tracking efficiently.
- Collected functional requirements (table booking, menu ordering, payment recording, admin monitoring) and non-functional requirements (performance, usability, scalability).

2. System Design

- Designed a three-layer architecture:
 - **User Interface Layer** (HTML, CSS, JavaScript frontend)
 - **Application Logic Layer** (Java Servlets handling booking, order, and payment logic)
 - **Data Storage Layer** (MySQL database for storing all system data).

3. System Development (Implementation)

- Implemented backend using Java Servlets.
- Developed modules for:
 - Table booking and availability validation
 - Food ordering and bill calculation
 - Payment storage and update
 - Admin dashboard for viewing bookings, orders, and payments
- Used MySQL for persistent storage of booking, food, and payment data.

4. System Integration

- Integrated booking, ordering, and payment modules into a unified system.
- Connected frontend pages with backend servlets using HTTP requests (Fetch API).
- Ensured proper communication between application logic and database operations..

5. System Testing

- Performed unit testing for individual modules (booking, ordering, payment, admin).
- Verified prevention of double booking logic.
- Tested database operations for accurate storage and retrieval of data.
- Checked admin dashboard for correct display of booking, food, and payment details.
- Fixed bugs related to data handling and user input validation.

6. System Maintenance

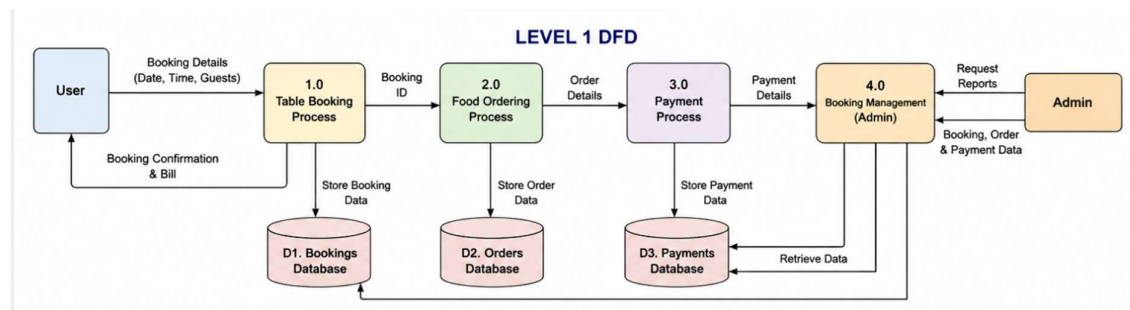
- Continuous improvement of system performance and UI.
- Bug fixing (booking errors, data mismatch, server issues).
- Optimization of database queries for faster response.
- Future updates include mobile compatibility, online payment integration, and enhanced UI design.

DATA FLOW DIAGRAM

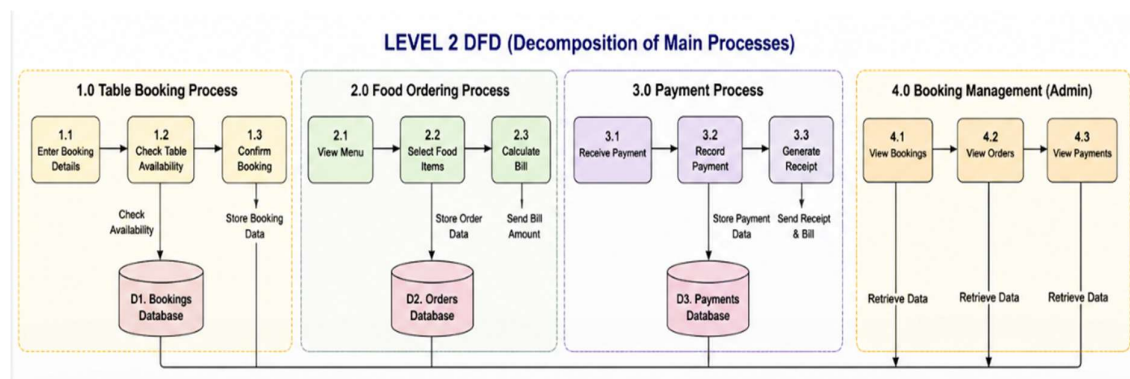
LEVEL 0 DFD



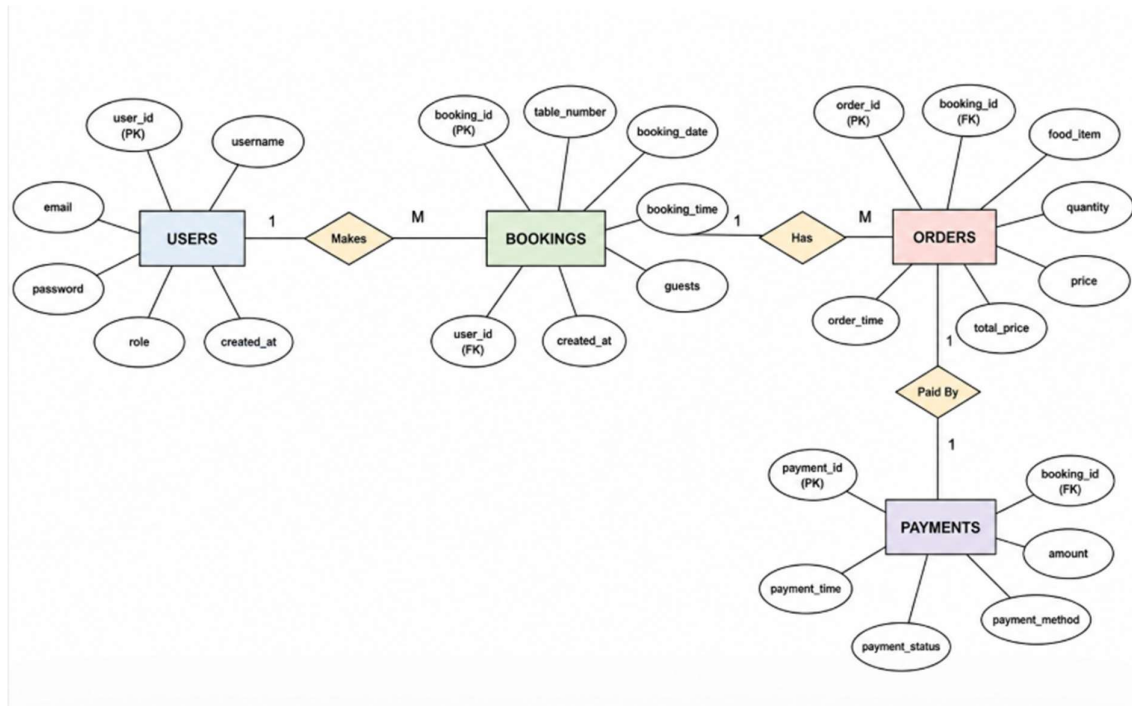
LEVEL 1 DFD



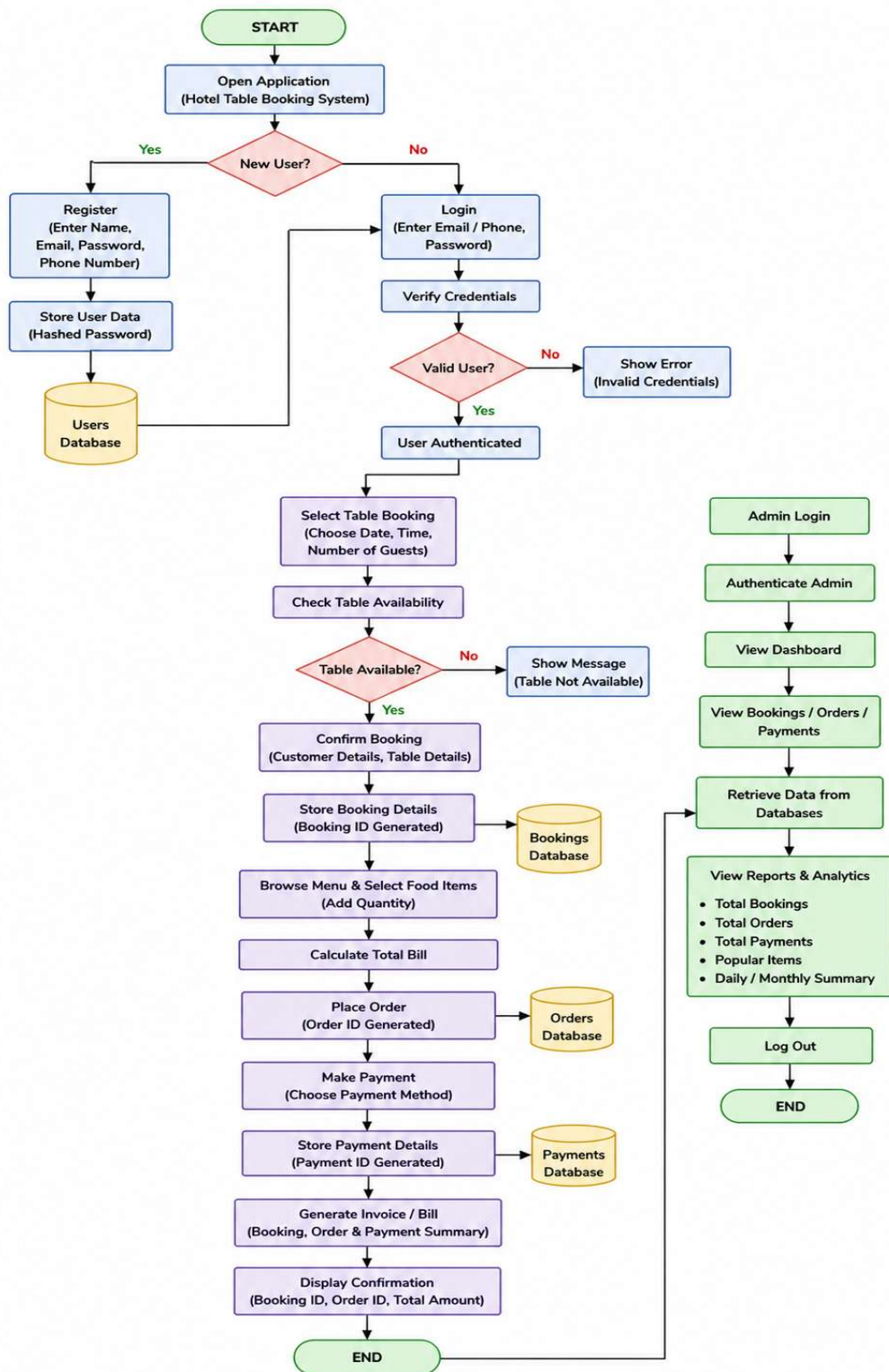
LEVEL 2 DFD



ER-DIAGRAM



FLOWCHART



SYSTEM DESIGN

1. Architectural Design

- **User Interface Layer:** Developed using HTML, CSS, and JavaScript, where users can book tables, order food, and make payments. Admin can log in and view booking details.
- **Application Logic Layer:** Implemented using Java Servlets (BookTableServlet, FoodServlet, PaymentServlet, ViewBookingsServlet). This layer handles booking validation, food ordering, payment processing, and data retrieval.
- **Data Storage Layer:** MySQL database is used to store all data including bookings, food orders, and payments in structured tables.

2. Data Flow Design

- **Input:** User enters details such as name, number of guests, date, time, and selects a table.
- **Processing:**
 - Check table availability (same date + time conflict)
 - Store booking details
 - Process food order and calculate bill
 - Store payment details
- **Output:**
 - Booking confirmation
 - Ordered food details
 - Total bill/payment status
 - Admin dashboard display

3. Data Storage Design

- **Bookings Table:** Stores booking information (*id, table_no, date, time, name, guests, food, food_status, payment*)
- **Orders (optional logic via same table):** Food items and status stored with booking
- **Payments:** Payment amount stored per booking

4. User Interface Design

- **Booking Page (index.html):** User selects table, date, time, and guests
- **Menu Page (menu.html):** User selects food items and quantity
- **Bill Page (bill.html):** Displays total amount and payment details
- **Admin Dashboard (admin.html):** Shows all bookings with:
 - Table number
 - Customer name
 - Food ordered
 - Status
 - Payment

MODULARISATION DETAILS

1. User Authentication Module

Routes:/auth/register,/auth/login

Functions:

- Register users and hash passwords
- Validate login credentials
- Generate JWT token

2. QR Scan Module

Route:/scan/qr

Functions:

- Accept QR data input
- Extract URL and domain
- Perform risk analysis
- Return risk score and level

3. Risk Analysis Module

Functions:

- Check HTTPS, URL length, keywords
- Detect suspicious patterns
- Calculate risk score and classification

4. Admin Module

Routes:/admin/scans,/admin/stats

Functions:

- View all scan records

- Generate system statistics
- Restrict access to admin users

5. Database Module

Functions:

- Connect to MySQL database
- Store users and scan data
- Retrieve scan history and stats

6. Session & Security Module

Technologies:JWT.Authentication

Functions:

- Maintain user sessions
- Protect routes
- Handle role-based access (user/admin)

DATA INTEGRITY AND CONSTRAINTS

1. Booking Module

Servlet: /BookTableServlet

Functions:

- Accept user input (name, guests, date, time, table)
- Check table availability (same date + time conflict)
- Prevent duplicate booking for same table
- Insert booking data into MySQL database
- Redirect user after successful booking

2. Table Management Module

Handled in: index.html + JS logic.

Functions :

- Display available tables dynamically
- Filter tables based on number of guests
- Mark already booked tables (using API data)
- Allow selection of best-fit table
- Store selected table (localStorage)

3. Food Ordering Module

Servlet: /FoodServlet

Functions:

- Accept selected food items and quantity
- Calculate total bill
- Update latest booking record with :
 - Food
 - Food_status = “Ordered”
- Store order details in database
- Send confirmation response to frontend

4. Payment Module

Servlet : /PaymentServlet

Function:

- Accept payment amount
- Update payment field in databse
- Link payment with latest booking (table-based)
- Confirm successful transaction

5. Booking View Module (Admin Module)

Servlet: /ViewBookingsServlet

Functions:

- Fetch all booking records from database
- Convert data into JSON format
- Send data to admin dashboard

- Display:
 - Table number
 - Customer details
 - Food status
 - Payment status

6. Order Retrieval Module

Servlet: /GetOrderServlet

Functions:

- Fetch latest food order and payment using table number
- Return JSON response (food + payment)
- Used for bill generation

7 . Database Module

Technology: MySQL

Functions:

- Store booking data (table_no, date, time, name, guests)
- Store food and order status
- Store payment details
- Perform CRUD operations using JDBC

8. Frontend Interaction Module

Technologies: HTML, CSS, JavaScript

Functions:

- **Handle user input forms**
- **Call backend servlets using fetch()**
- **Display booking, menu, and bill UI**
- **Manage localStorage (table, bill)**

DATABASE DESIGN

1. Database Name:

cafe_db

2. Table Structure

a) bookings table

Stores all table reservations along with food and payment info.

SQL:

```
CREATE TABLE bookings(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    table_no VARCHAR(10) NOT NULL,  
    date DATE NOT NULL,  
    time TIME NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    guests INT NOT NULL,  
    food TEXT,  
    food_status VARCHAR(50),  
    payment INT DEFAULT 0  
);
```

b) users table

For admin/user login (if you extend system)

SQL :

```
CREATE TABLE users(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE,  
    password VARCHAR(100)  
);
```

3. RELATIONSHIPS

a) One-to-Many:

One table → multiple bookings over time

(table_no reused across different dates/times)

b) Booking → Food → Payment (Single Flow):

One booking contains:

- Food details
- Food status
- Payment

USER INTERFACE DESIGN

BOOKING PAGE

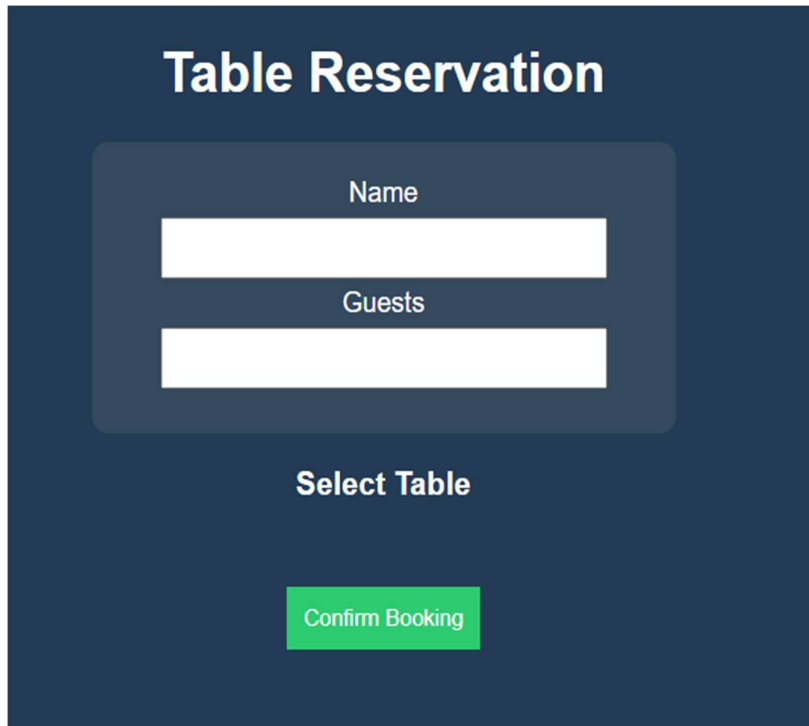


Table Reservation

Name

Guests

Select Table

Confirm Booking

TABLE SELECTION

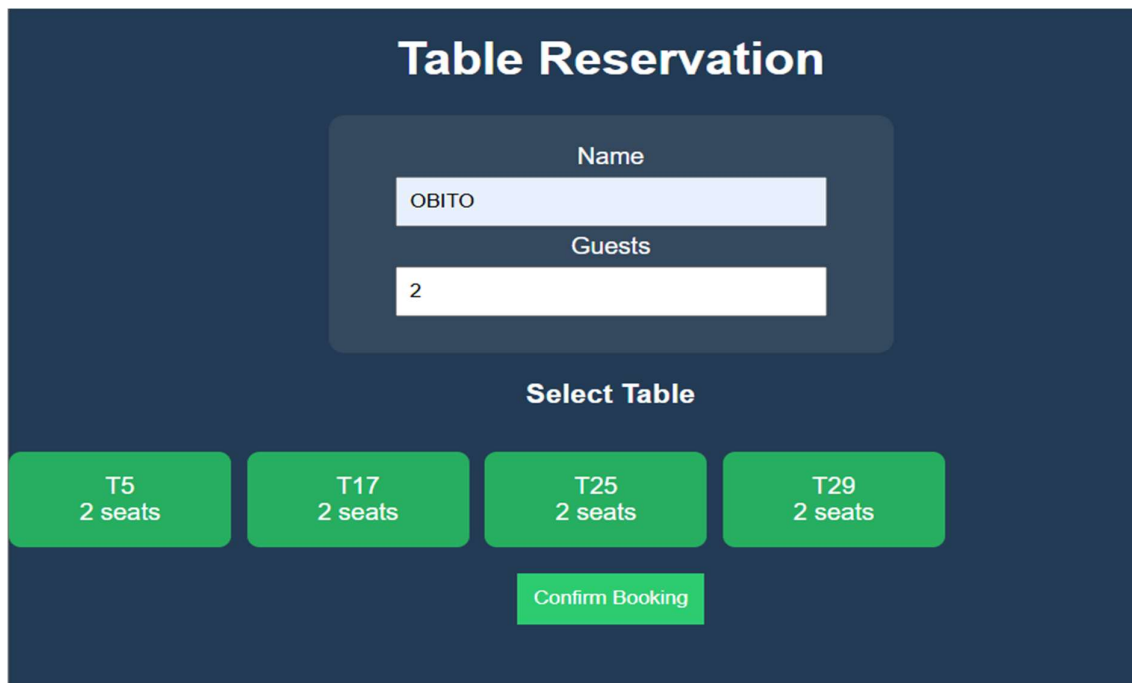


Table Reservation

Name

Guests

Select Table

T5
2 seats

T17
2 seats

T25
2 seats

T29
2 seats

Confirm Booking

MENU PAGE

 **Full Menu**



Paneer Butter Masala ₹250



Palak Paneer ₹230



Dal ₹150



Chicken Curry ₹280



Tandoori ₹320



Veg Burger ₹120

BILL / PAYMENT PAGE

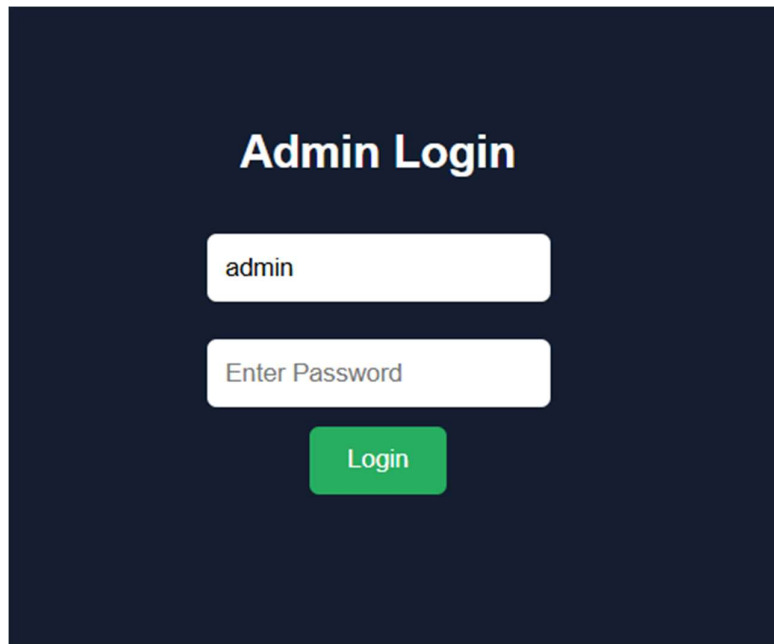
Payment Gateway

Amount ₹250

Select Method 

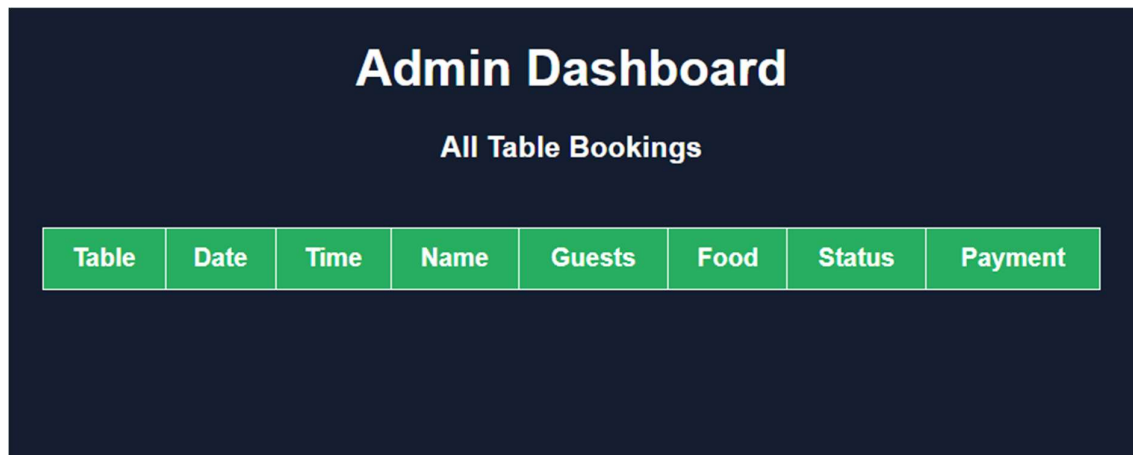
Pay Now

ADMIN LOGIN PAGE



The Admin Login form is centered on a dark blue background. It features the title "Admin Login" in white. Below the title are two white input fields: the first contains the text "admin", and the second contains the placeholder text "Enter Password". Below these fields is a green "Login" button with white text.

ADMIN DASHBOARD



The Admin Dashboard section has a dark blue background. It features the title "Admin Dashboard" in white, followed by the subtitle "All Table Bookings" in white. Below the subtitle is a table with 8 columns, each with a green header and white text.

Table	Date	Time	Name	Guests	Food	Status	Payment
-------	------	------	------	--------	------	--------	---------

TESTING

1. Testing Techniques and Testing Strategies Used

To ensure the reliability and correctness of the **Hotel Table Booking Reservation System**, the following testing techniques were applied :

a) Unit Testing

- Individual modules such as:
 - Booking (BookTableServlet)
 - Food ordering (FoodServlet)
 - Payment processing (PaymentServlet)
 - Data fetching (ViewBookingsServlet)
- Each servlet was tested independently to verify correct functionality.

b) Integration Testing

- Verified interaction between modules:
 - Booking → Database
 - Booking → Food order
 - Food order → Payment
 - Database → Admin Dashboard
- Ensured proper data flow between frontend and backend.

c) System Testing

- Tested the complete system workflow :
 - Table booking → Food order → Payment → Admin view
- Checked:
 - End-to-end booking process
 - Data storage and retrieval
 - UI updates

d) API / Servlet Testing

- Tested servlet URLs using browser / Postman :
 - /BookTableServlet
 - /FoodServlet
 - /PaymentServlet
 - /ViewBookingsServlet
- Verified request/response correctness.

e) Validation & Error Testing

- Checked:
 - Empty input fields
 - Invalid data(no table selected)
 - Duplicate booking (same table , date, time)
- Ensured system show proper error messages .

2. Test Reports for Unit Test Cases and System Test Cases

a)Unit test cases

Test case	Module	Input	Expected Output	Status
Book Table	Booking	Valid data	Data inserted in DB	Pass
Duplicate Booking	Booking	Same table/date/time	Booking rejected	Pass
Food Order	Food	Valid items	Food updated	Pass
Payment	Payment	Amount	Payment stored	Pass
View Bookings	Admin	Request data	JSON data returned	Pass

b) System test cases

Test case	Test Description	Expected Output	Status
Full Flow test	Booking to Food to Payment	Complete process works	Pass
Admin Access	Open dashboard	Data displayed correctly	Pass
Data Storage	Book table	Data stored in DB	Pass
Error Handling	Missing input	Error message shown	Pass
Security Check	Same table/time	Booking blocked	Pass

SYSTEM SECURITY MEASURES

1.Authentication & Authorization

- Admin login system is implemented using username and password.
- Unauthorized users cannot view booking data.
- Only authorized admin can access the dashboard.

2. Input Validation

- User inputs are validated before processing:
 - Name and guests must not be empty
 - Date and time must be selected
 - Table must be selected
- Prevents invalid or incomplete bookings.

3. Duplicate Booking Prevention

- System checks:
 - Same **table + date + time**
- If already booked:
 - Booking is rejected
 - User is notified

4. Secure Database Access

- MySQL database accessed using JDBC connection
- SQL queries executed using **PreparedStatement**
- Prevents: SQL Injection attacks

5. Data Integrity

- Each booking has a unique **id (Primary Key)**
- Booking data stored consistently:
 - table_no
 - date
 - time
 - food
 - payment
- Ensures reliable data storage and retrieval.

6. Error Handling & Access Control

- Exceptions handled using try-catch blocks
- Server errors are not exposed to users
- User gets simple error messages instead of technical details

7. Controlled Data Access (Admin Panel)

- Admin dashboard fetches data via servlet
- Only backend controls what data is shown
- Prevents direct database manipulation

FUTURE SCOPE AND FURTHER ENHANCEMENT

Short-term enhancements:

- Improve UI/UX of booking and menu pages for better user experience.
- Add **real-time table availability** display.
- Enable **order modification** (update or cancel food orders).
- Show **live bill calculation** before confirmation.
- Add **validation alerts** for better error handling.

Medium-term enhancements:

- Implement **online payment integration** (UPI, card, wallet).
- Add **user login system** for booking history tracking.
- Introduce **kitchen dashboard** to manage incoming food orders.
- Enable **notifications system** (order placed, ready, payment done).
- Add **search and filter options** in admin dashboard.

Long-term enhancements:

- Develop **mobile application (Android/iOS)** for customers and admin.
- Implement **QR-based table ordering system** (scan → order directly).
- Use **AI-based recommendation system** for suggesting popular dishes.
- Deploy system on **cloud server** for scalability and remote access.
- Expand into a **complete restaurant management system** including:
 - Inventory management
 - staff management
 - sales analytics

CODING

Frontend Implementation (HTML, CSS, JavaScript)

(a) Booking Page (index.html)

```
<form action="BookTableServlet" method="post">
<input type="text" name="name" placeholder="Name">
<input type="number" name="guests" placeholder="Guests">
<input type="date" name="date">
<input type="time" name="time">
<input type="hidden" name="table" id="tableInput">
<button type="submit">Confirm Booking</button>
</form>
```

(b) Menu Page (menu.html)

```
fetch("/HotelTableReservation/FoodServlet",{
method:"POST",
headers:{
"Content-Type":"application/x-www-form-urlencoded"
},
body:`table=${table}&food=${items.join(", ")}&amount=${total}`
});
```

(c) Admin Dashboard (admin.html)

```
fetch("/HotelTableReservation/ViewBookingsServlet")
.then(res=>res.json())
.then(data=>{
data.forEach(row=>{
console.log(row.table,row.food,row.payment);
});
});
```

Backend Implementation (Java – Servlets)

(a) Booking Module – BookTableServlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class BookTableServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String name = req.getParameter("name");
        String guests = req.getParameter("guests");
        String date = req.getParameter("date");
        String time = req.getParameter("time");
        String table = req.getParameter("table");
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/cafe_db", "root", "");
            PreparedStatement check = con.prepareStatement(
                "SELECT * FROM bookings WHERE table_no=? AND date=? AND time=?");
            check.setString(1, table);
            check.setString(2, date);
            check.setString(3, time);
            ResultSet rs = check.executeQuery();
            if (rs.next()) {
                res.getWriter().println("BOOKED");
                return;
            }
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO bookings(table_no,date,time,name,guests) VALUES(?,?,?,?,?)");
            ps.setString(1, table);
            ps.setString(2, date);
            ps.setString(3, time);
            ps.setString(4, name);
            ps.setInt(5, Integer.parseInt(guests));
            ps.executeUpdate();
            res.getWriter().println("SUCCESS");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```


(b) Food Module – FoodServlet

```
public class FoodServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String table = req.getParameter("table");
        String food = req.getParameter("food");
        int amount = Integer.parseInt(req.getParameter("amount"));

        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/cafe_db","root","");

            PreparedStatement ps = con.prepareStatement(
                "UPDATE bookings SET food=?, food_status='Ordered', payment=? WHERE table_no=?
ORDER BY id DESC LIMIT 1");

            ps.setString(1, food);
            ps.setInt(2, amount);
            ps.setString(3, table);

            ps.executeUpdate();

            res.getWriter().println("ORDERED");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

(c) Payment Module – PaymentServlet

```
public class PaymentServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String table = req.getParameter("table");
        int amount = Integer.parseInt(req.getParameter("amount"));

        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/cafe_db","root","");

            PreparedStatement ps = con.prepareStatement(
                "UPDATE bookings SET payment=? WHERE table_no=? ORDER BY id DESC LIMIT 1");

            ps.setInt(1, amount);
            ps.setString(2, table);
```

```

        ps.executeUpdate();

        res.getWriter().println("PAID");

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

(d) Admin Module – ViewBookingsServlet

```

public class ViewBookingsServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        res.setContentType("application/json");
        PrintWriter out = res.getWriter();

        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/cafe_db","root","");

            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM bookings");

            StringBuilder json = new StringBuilder("[");
            boolean first = true;

            while(rs.next()){

                if(!first) json.append(",");
                first = false;

                json.append("{");
                json.append("\"table\":").append(rs.getString("table_no")).append(",");
                json.append("\"name\":").append(rs.getString("name")).append(",");
                json.append("\"food\":").append(rs.getString("food")).append(",");
                json.append("\"payment\":").append(rs.getInt("payment")).append(",");
                json.append("}");

            }

            json.append("]");
            out.print(json.toString());

        } catch (Exception e){
            e.printStackTrace();
        }
    }
}

```

Output:

Table Reservation

Name

Guests

Select Table

Confirm Booking

Table Reservation

Name

Guests

Select Table

T5
2 seats

T17
2 seats

T25
2 seats

T29
2 seats

Confirm Booking

Full Menu

 Paneer Butter Masala ₹250 01	 Palak Paneer ₹230 0	 Dal ₹150 0
 Chicken Curry ₹280 0	 Tandoori ₹320 0	 Veg Burger ₹120 0

Payment Gateway

Amount ₹250

Select Method ▼

Pay Now

Admin Login

admin

Enter Password

Login

Admin Dashboard

All Table Bookings

Table	Date	Time	Name	Guests	Food	Status	Payment
-------	------	------	------	--------	------	--------	---------

BIBLIOGRAPHY & PROFILES OF TWO IT COMPANIES

Bibliography:

- Java Servlet Documentation — [docs.oracle.com/javase](https://docs.oracle.com/javase/7/docs/api/)
- MySQL Reference Manual — [dev.mysql.com](https://dev.mysql.com/doc/)
- Apache Tomcat Documentation — tomcat.apache.org
- HTML, CSS, JavaScript Guide — developer.mozilla.org
- JDBC (Java Database Connectivity) Guide — [docs.oracle.com/javase](https://docs.oracle.com/javase/7/docs/api/java/sql/JDBC-4.2/)
- W3Schools Web Development Tutorials — www.w3schools.com
- Oracle Java Documentation — [docs.oracle.com/javase](https://docs.oracle.com/javase/7/docs/api/)
- Stack Overflow — www.stackoverflow.com

Profiles of Two IT Companies:

1. Company A — Infosys

a) Basic Information & Company Overview

- Infosys Ltd., founded 1981, global IT services and consulting company headquartered in India.

b) Services and Solutions

- Enterprise application development, cloud & infrastructure, data engineering, software QA, digital experience.

c) Technologies and Expertise

- Extensive experience with Python, web frameworks, databases (MySQL, PostgreSQL), cloud (AWS/Azure).

d) Careers and Work Culture

- Structured onboarding, continuous learning, campus placement programs.

e) Placement Process

- Eligibility: Graduates with required CGPA; selection rounds typically include aptitude, technical interview, HR round.

f) Corporate Social Responsibility (CSR)

- Education initiatives, skill development programs, sustainability projects.

g) Contact Information

- Public site: <https://www.infosys.com> (use company's careers page for contact, local office addresses vary by country).

Company B — Tata Consultancy Services (TCS)

a) Basic Info & Overview

- TCS is a global IT services, consulting and business solutions organization based in India.

b) Services and Solutions

- Application development, cloud solutions, data analytics, enterprise systems.

c) Technologies and Expertise

- Works extensively with full-stack tech, Python, Java, databases, enterprise architecture.

d) Careers and Work Culture

- Strong focus on employee development, reskilling, and career growth.

e) Placement Process

- Written test (aptitude), technical interview, HR.

f) **CSR**

- Programs in education, healthcare, and sustainability.

g) **Contact Information**

- <https://www.tcs.com> (see careers for recruitment info)